

WELCOME

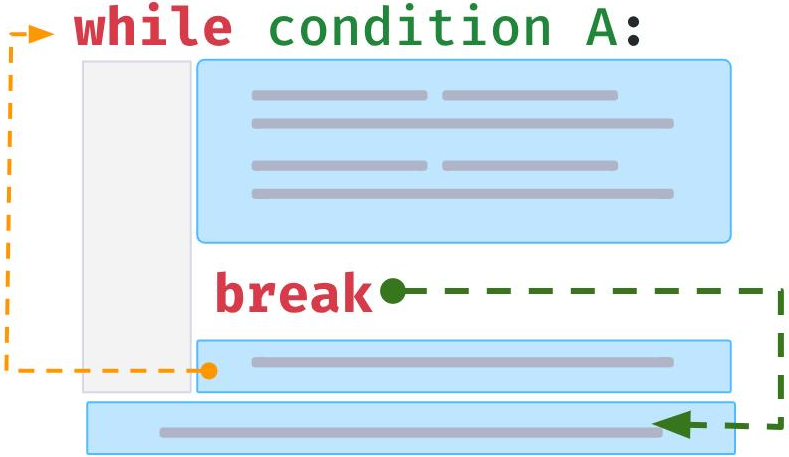
CCBP 4.0

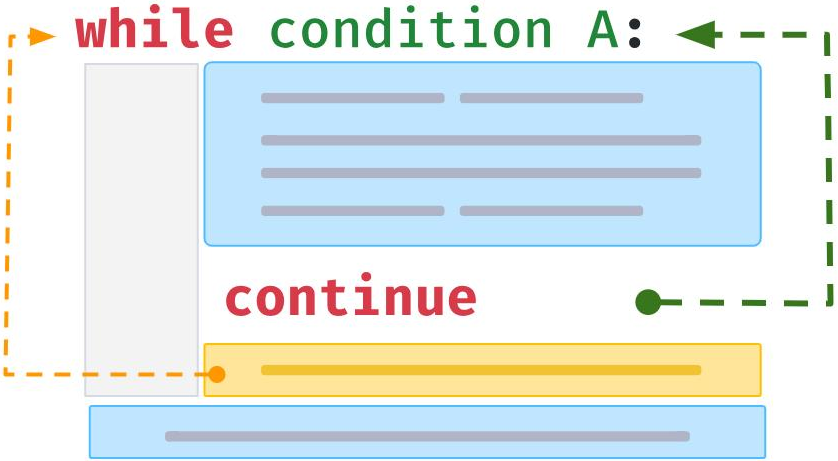
21st Century Skills

Comparing Strings & Naming Variables

Recap

Loop Control Statements





Recap

Pass

Pass statement is used as a syntactic placeholder.

When it is executed, nothing happens

Generally used when
we have to test the code
before writing the complete code

```
if condition A:
    Block 1
elif condition B:
    pass
else:
    Block 3
```

Daily Challenge

Identify The Mistake

Code

```
n = int(input())
is_prime = True
for i in range(2, n):
    if (i % n) == 0:
        is_prime = False
        break
print(is_prime)
```

Check whether the given number is a prime number

Input

```
22
```

Expected Output

```
False
```

NXT WAVE

Daily Challenge

Identify The Mistake

Code

```
n = int(input())
is_prime = True
for i in range(2, n):
    if (n % i) == 0:
        is_prime = False
        break
print(is_prime)
```

Input

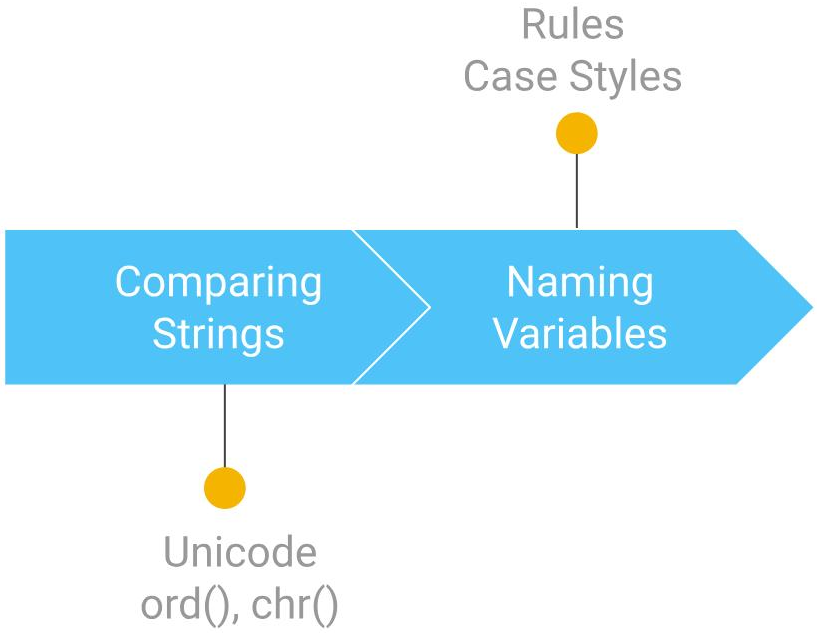
```
22
```

Output

```
False
```

Agenda for Today's Session

Comparing Strings & Naming Variables



Python

Unicode

Computer internally stores characters as numbers.

Every character has a unique Unicode value.

"A"

Unicode - 65

"z"

Unicode - 122

"1"

Unicode - 49

"*"

Unicode - 42

NXT WAVE

Finding Unicode Value

To find the Unicode value of a character, we use the `ord()`

```
ord(character)
```



Gives unicode value for this character

Finding Unicode

Ord

Code

```
unicode_value = ord("A")
print(unicode_value)
```

Output

```
65
```

Character at Unicode

To find the character with the given Unicode value,

we use the `chr()`

```
chr(unicode)
```



Gives character with the unicode value

Finding Char

Character at Unicode

Code

```
char = chr(75)
print(char)
```

Output

```
K
```

Code

```
char = chr(100)
print(char)
```

Output

```
d
```

Unicode Ranges

48 - 57	→	Number Digits (0 - 9)
65 - 90	→	Capital Letters (A - Z)
97 - 122	→	Small Letters (a - z)
Rest	→	Special Characters, Other Languages

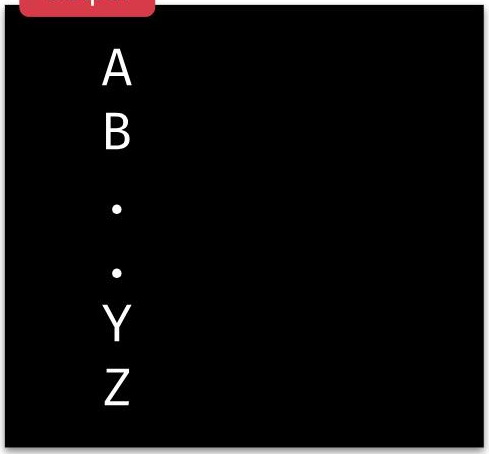
Finding Char

Printing Characters

Code

```
for unicode_value in range(65,91):
    print(chr(unicode_value))
```

Output



Comparing Strings

In Python,

strings are compared

considering unicode

Code

```
print("A" < "B")
```

Comparing Strings

Code

```
print("A" < "B")
```

Output

```
True
```

Comparing Strings

Code

```
print("a" > "A")
```

Code

```
print("9" > "5")
```

Comparing Strings

Code

```
print("a" > "A")
```

Output

```
True
```

Code

```
print("9" > "5")
```

Output

```
True
```

Comparing Strings

In Python,

String Comparison is done

character by character

Code

```
print("BAD" >= "BAT")
```

Comparing Strings

Code

```
print("BAD" >= "BAT")
```

Output

```
False
```

Comparing Strings

Code

```
print("98" <= "123")
```

Comparing Strings

Code

```
print("98" <= "123")
```

Output

```
False
```

Comparing Strings

Code

```
print("98" < "984")
```

Comparing Strings

Code

```
print("98" < "984")
```

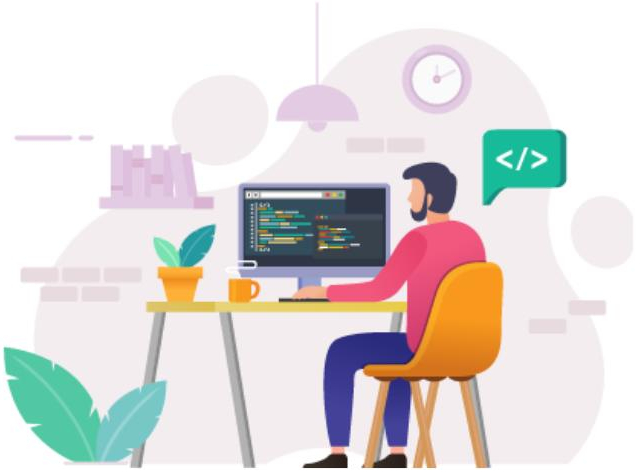
Output

```
True
```

Naming Variables

Rules for Naming Variables

Let's look into rules for naming Python variables



Naming Variables

Rule #1

- We can use
  - Capital Letters (`A – Z`)
  - Small Letters (`a – z`)
  - Digits (`0 – 9`)
  - Underscore(`_`)

Naming Variables

Rule #1

- We can use
  - Capital Letters (`A – Z`)
  - Small Letters (`a – z`)
  - Digits (`0 – 9`)
  - Underscore(`_`)

`age` ✓ `input_1` ✓

`high_score` ✓ `_counter2` ✓

Naming Variables

Rule #2

- We cannot use
  - Spaces ( )
  - Commas (,)
  - Special Characters

    (~ ! @ # $ % ^ . ?, etc..)

Naming Variables

Rule #2

- We cannot use
  - Spaces ( )
  - Commas (,)
  - Special Characters
    (~ ! @ # $ % ^ . ?, etc..)

`left` ✓

`start_playing` ✓

`read, write & learn` ✗

Naming Variables

Rule #3

- Variable name must begin with
  - Capital Letter (`A – Z`)
  - Small Letter (`a – z`)
  - Underscore(`_`)

Naming Variables

Rule #3

- Variable name must begin with
  - Capital Letter (`A – Z`)
  - Small Letter (`a – z`)
  - Underscore(`_`)

`left` ✓ `_player1` ✓

`Height_in_feet` ✓ `1_player` ✗

Naming Variables

Rule #4

Cannot use **Keywords,**

which are reserved for special meaning

in Python like `while,` etc.

Naming Variables

Rule #4

Cannot use **Keywords,**

which are reserved for special meaning

in Python like `while,` etc.

`move` ✓ `_player1` ✓

`_if` ✓ `while` ✗

Naming Variables

Keywords

Reserved Keywords								
False	class	from	or	None	continue	global	pass	break
True	def	if	raise	and	nonlocal	import	return	for
as	elif	in	try	assert	else	is	while	not
async	except	lambda	with	await	finally	del	yield	

Code

```
help("keywords")
```

Identify valid variables

1) `User_name`
2) `in`
3) `(yes no)?`
4) `1_age`
5) `_player_age`

Naming Variables

Identify valid variables

1) `User_name` 
2) `in`
3) `(yes no)?`
4) `1_age`
5) `_player_age`

Naming Variables

Identify valid variables

1) `User_name` 

2) `in`  *Should not be a keyword*

3) `(yes no)?`

4) `1_age`

5) `_player_age`

Naming Variables

Identify valid variables

1) `User_name` 

2) `in`  *Should not be a keyword*

3) `(yes no)?`  *Spaces and Special characters are not allowed*

4) `1_age`

5) `_player_age`

Identify valid variables

1) `User_name` 

2) `in`  *Should not be a keyword*

3) `(yes no)?`  *Spaces and Special characters are not allowed*

4) `1_age`  *Must begin with a character or an underscore*

5) `_player_age`

Identify valid variables

1) `User_name` 

2) `in`  *Should not be a keyword*

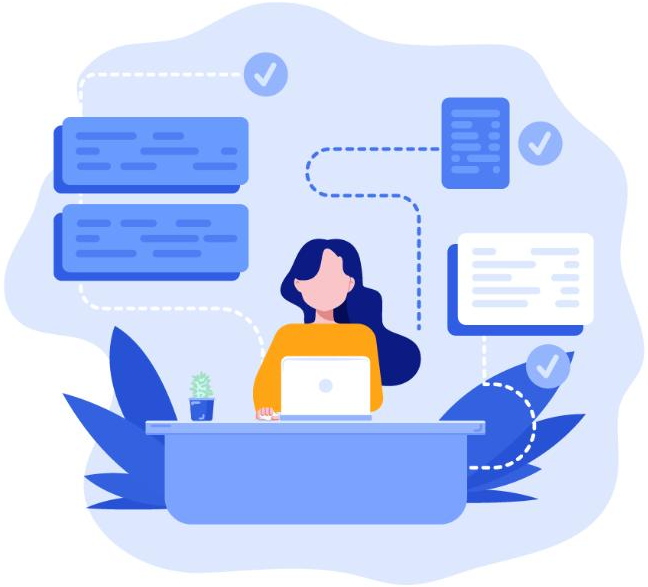
3) `(yes no)?`  *Spaces and Special characters are not allowed*

4) `1_age`  *Must begin with a letter or an underscore*

5) `_player_age` 

Naming Conventions

Good naming conventions
can help readers
understand the program easily



Naming Conventions

Let’s learn about most common naming conventions

Case Styles



Naming Conventions

Case Styles

Variable: Total Bill

Camel case: **`totalBill`**

Case Styles

Variable: Total Bill

Camel case: **`totalBill`**

Pascal case: **`TotalBill`**

Naming Conventions

Case Styles

Variable: Total Bill

Camel case: **`totalBill`**

Pascal case: **`TotalBill`**

Snake case: **`total_bill`**

Case Styles

In Python programming,
We follow snake case notation
for naming variables

Snake case: **`total_bill`**

Question

Lets store the sum of even numbers into a variable

Which among the following names is most suitable?

- `sum`
- `sumofevenums`
- `sum_of_even_numbers`
- `sumofevennumbers`

Naming Variables

Question

Lets store the sum of even numbers into a variable

Which among the following names is most suitable?

- `sum`
- `sumofevenums`
- `sum_of_even_numbers` 
- `sumofevennumbers`

Best Practices

Key Takeaways

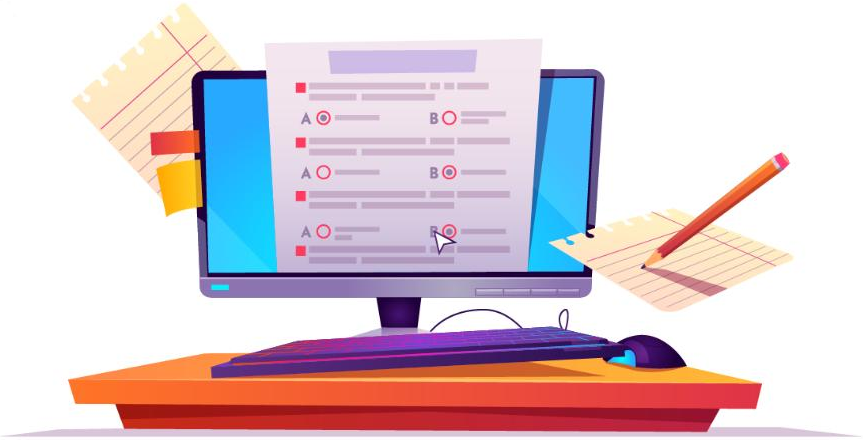
- Comparing Strings
  - Unicode
  - `ord()` , `chr()`
- Rules for Naming Variables
- Case Styles



Next Session

Code Walkthrough

Practice



Feedback



NXT WAVE

THANK YOU



ALL THE BEST